



Distributed System Project & Written Report

Distributed Systems - CSI3344.4
Assignment – 02

Himasha Malith Amarasinghe - 10660238
Lecturer – Mr. Dammike Mendis

Executive Summary

This document outlines the design, setup, and testing of a three-level distributed banking setup built to illustrate key ideas in distributed computing. Core functions like logging in, checking balances, or moving funds were included along with varying cost rules across levels. Dealing with common issues - like ensuring data matches everywhere, handling failures, or connecting from faraway locations - was central to its purpose.

The project was developed in two phases after following recommended iterative methodology. Phase 1 created a two-tier architecture using in-memory storage to test business logic and communications patterns using Pyro5 RPC framework. Phase 2 was the extension of the system into a full three-tier architecture via a dedicated database server (BDB Server) using SQLite for persistent storage, providing complete separation of concerns between presentation (BC Client), business logic (BAS Server) and data persistence (BDB Server) layers.

The system achieves its main technical goals through synchronous request-response communication which supports financial operations that need instant confirmation and a fee calculation system that progresses through six levels. Starting from free transfers under \$2,000 to 0.05% fees for transfers above \$100,000 (with per-tier caps). The system uses transfer ID verification to handle duplicate requests effectively while providing simple duplicate request management. The system achieves complete observability through its implementation of structured logging and its ability to export data in CSV format.

The system proves its ability to follow ACID transaction rules through SQLite transaction management, and it upholds critical banking operation consistency standards and offers robust error handling with useful client feedback. The testing process confirms that all functional requirements have been met because it checks authentication processes and fee computation for different user levels and it also tests all possible scenarios including when users run out of money and when they try to send money to incorrect addresses and it checks how the system stores data through multiple server shutdowns.

The system design implements academic learning targets through practical distributed systems principles which enable students to understand how to separate system tiers and perform remote procedure calls and handle transactions while learning about distributed system tradeoffs between consistency and availability and partition tolerance.

Table of Contents

1.	Introduction	3
2.	Application and Technology Background.....	3
2.1	Banking Systems as Distributed Systems	3
2.2	Technology Stack Selection	4
2.3	Synchronous RPC Implementation	4
3.	Application Requirements.....	4
3.1	Functional Requirements.....	4
3.2	Transfer Fee Structure	5
3.3	Non-Functional Requirements.....	5
3.4	Mock Data and Testing Context.....	6
4.	System Design and Architecture	6
4.1	Three-Tier Architecture Overview	6
4.2	Component Communication Patterns.....	7
4.3	Authentication and Session Management	7
4.4	Transfer Processing Workflow.....	9
4.5	Database Schema and Persistence	10
4.6	Consistency and Transaction Management	11
4.7	Duplicate Request Handling	11
5.	Implementation Procedure.....	12
5.1	Phase 1: Two-Tier Development	12
5.2	Phase 2: Three-Tier Extension with Database	12
5.3	Development Challenges and Solutions.....	13
6.	User Manual	14
6.1	System Requirements	14
6.2	Installation and Setup	14
6.3	Starting the System.....	15
6.4	Using the Banking System.....	16
7.	Test Cases and Validation.....	19
8.	Discussion and Critical Analysis.....	22
8.1	Design Decisions and Justifications.....	22
8.2	Trade-offs and Limitations	22
8.3	Lessons Learned	22
9.	Conclusion	22
10.	References	23

LIST OF FIGURES

FIGURE 1: FEE STRUCTURE TABLE.....	5
FIGURE 2: THREE-TIER ARCHITECTURE	7
FIGURE 3: TRANSFER PROCESSING WORKFLOW.....	9
FIGURE 4: PROJECT FOLDER STRUCTURE	14
FIGURE 5: TERMINAL 1	15
FIGURE 6: TERMINAL 2	15
FIGURE 7: TERMINAL 3	15
FIGURE 8: TERMINAL 4	16
FIGURE 9: SUCCESSFUL LOGIN SCREEN SHOWING SESSION TOKEN	16
FIGURE 10: VIEW BALANCE	17
FIGURE 11: SUCCESSFUL TRANSFER SHOWING AMOUNT, FEE, AND TOTAL DEDUCTED	17
FIGURE 12: CHECKING TRANSFER STATUS.....	18
FIGURE 13: CSV EXPORT CONFIRMATION MESSAGE	18
FIGURE 14: ERROR MESSAGE FOR INVALID CREDENTIALS	19
FIGURE 15: INVALID RECIPIENT.....	21

1. Introduction

Modern computing infrastructure depends on distributed systems which provide scalable and reliable networked applications through their fundamental structure. The banking sector faces major challenges when implementing distributed computing because it needs to preserve both data consistency and transactional integrity and system fault tolerance at the same time. The assignment creates a basic distributed banking system which shows essential distributed systems principles through its implementation of remote procedure calls (RPC) and its use of tier separation and transaction management and consistency guarantees.

The banking transfer system enables users to authenticate themselves while they can check their account balances and perform fund transfers between different accounts. The system architecture and design principles of this educational example use mock data to create a simplified version which resembles actual distributed banking systems found in real-world environments. The system solution solves fundamental distributed systems problems which involve handling distant system interactions and handling data between different layers and preventing duplicate requests and ensuring system consistency during system failures.

The implemented system uses Python 3.8+ as the development platform with Pyro5 (Python Remote Objects) for inter-process communication and SQLite for data persistence. The technology stack shows educational simplicity because it uses distributed systems principles which demonstrate enough complexity to show real-world system operation. The following sections present the technical background and system architecture and implementation approach and validation methodology which guided the entire project.

2. Application and Technology Background

2.1 Banking Systems as Distributed Systems

The banking transfer system functions as a distributed system because it includes various client interfaces such as web and mobile and ATM systems and application servers that

execute business logic and database servers which store persistent information (Tanenbaum & Van Steen, 2017). The current banking system design separates different functions between its layers to provide both performance improvement and system stability and protection of sensitive information. The client tier performs user interface functions, but the application tier enforces business rules and authentication tasks, and the data tier maintains permanent storage with built-in transactional protection.

2.2 Technology Stack Selection

The system depends on Pyro5 (Python Remote Objects) as its RPC framework to establish process-to-process communication. Pyro5 enables users to perform remote method invocation without requiring explicit code because distributed objects can be accessed through Python object interfaces (Pyro5 Documentation, 2024). The system development process becomes more accessible because it conceals network communication complexities which let developers handle remote system boundaries directly. The academic teaching system benefits from Pyro5 because it provides a Pythonic API and minimal boilerplate code which enables students to reach their learning targets while demonstrating RPC principles effectively.

The persistence layer of SQLite operates as a database system which guarantees ACID transactions for banking operations according to the SQLite Consortium (2024). The standard production banking systems use client-server databases which include PostgreSQL and MySQL but SQLite offers embedded deployment and testing capabilities with complete SQL functionality and transaction control. The system architecture maintains SQLite operation within BDB Server boundaries while preserving database tier separation which applies to all database management systems.

2.3 Synchronous RPC Implementation

The system operates through synchronous RPC because clients need to wait for server responses before continuing their execution. Financial operations need instant confirmation because customers need to know right away about the success or failure of their \$10,000 transfer. Synchronous RPC functions through request-response semantics which provides fundamental error handling capabilities and maintains simple transaction limit management.

3. Application Requirements

3.1 Functional Requirements

The banking system requires authenticated sessions to enable users to perform their four essential banking operations.

FR1: User Authentication - The system requires users to authenticate through their username and password combination which produces session tokens for future system access. The system needs to preserve authentication state from start to finish of each session while it executes all operations to verify the presence of an active token.

FR2: Balance Query - Authenticated users must be able to access their current account balances through synchronous requests. The balanced information system requires a display which shows all completed transactions along with their respective fees.

FR3: Money Transfer - Users must be able to use a transfer system which lets them send funds between accounts by entering the recipient's username and transfer amount and an optional message. The system will determine fees through its tiered system while it checks for enough available funds (the transfer amount plus the fee) and blocks all self-transfer operations and checks if the recipient account exists before performing an atomic transfer operation.

FR4: Transfer Status Query - Users must be able to access transfer status by using transfer IDs to obtain all transfer information which includes sender and recipient details and amount data and fee amounts and timestamp and completion status.

3.2 Transfer Fee Structure

The system implements a progressive fee structure incentivizing smaller transfers while capping fees on larger amounts:

Transfer amount per transfer	Percentage on this transfer	Per-transfer cap	Notes
\$0 – \$2,000.00	0%	—	Free tier
\$2,000.01 – \$10,000.00	0.25%	\$20.00	Entry tier
\$10,000.01 – \$20,000.00	0.20%	\$25.00	Mid tier
\$20,000.01 – \$50,000.00	0.125%	\$40.00	Upper-mid tier
\$50,000.01 – \$100,000.00	0.08%	\$50.00	High tier
\$100,000.01 and above	0.05%	\$100.00	Top tier

Figure 1: Fee Structure Table

As an example: a \$5,000 transfer fee equals \$12.50 because it represents 0.25% of the transfer amount. The transfer fee for \$150,000 amounts to \$75.00 because it represents 0.05% of the transfer amount which falls under \$100 cap. All monetary values are round to two decimal places to prevent precision errors.

3.3 Non-Functional Requirements

NFR1: Consistency - The system requires absolute consistency to preserve correct account balance information. The creation of money and its destruction and duplication must never occur. The system requires all transfer operations to perform as single units of work which will either execute both debit and credit operations or none at all.

NFR2: Tier Separation - The system architecture needs to separate all system concerns into three distinct tiers which must operate independently from each other. The BC Client shall not access the database or BDB Server. The BAS Server shall not directly access the database file. Only the BDB Server may import sqlite3 and execute SQL statements.

NFR3: Duplicate Request Handling - The system requires duplicate transfer request identification and processing because network timeouts cause clients to send duplicate requests. The system includes a mechanism which stops duplicate requests from starting new transfer operations.

NFR4: Persistence and Auditability - The system requires all transfer operations to be stored through a recording system which includes sender ID and recipient ID and amount and fee and

timestamp and status information. The system needs to enable CSV export functionality, which allows users to export all database tables for audit requirements.

NFR5: Error Handling - The system needs to generate useful error messages which should appear for every system failure that includes authentication problems and insufficient funds and wrong recipient addresses and server connection breakdowns. System failure mode identification represents a fundamental requirement which clients need to fulfill.

3.4 Mock Data and Testing Context

This is an academic teaching system using mock data. Two pre-configured users exist:

Username "alice" (password: pass123, initial balance: \$10,000.00)

Username "bob" (password: pass456, initial balance: \$5,000.00)

The system does not have any functionality which enables it to connect with operational banking systems or payment processing networks. Production systems require password storage in plain text for educational purposes but this method fails to meet production system security standards. The system design focuses on distributed system architecture and coordination methods instead of delivering security features that are ready for production use.

4. System Design and Architecture

4.1 Three-Tier Architecture Overview

The system uses a three-tier architecture which maintains separate layers for presentation and business logic and data storage functions.

Tier 1: BC Client (Banking Client) - The client tier enables users to access a text-based command-line interface which they use to perform their interactions. It connects exclusively to the BAS Server via Pyro5 RPC, never directly accessing the database or BDB Server. The client contains no business logic; it delegates all operations to the application tier. After login, a session token is issued and used to authenticate subsequent requests.

Tier 2: BAS Server (Bank Application Server) - The application tier contains all business logic which includes authentication and session management and transfer fee calculation through the tiered structure and transfer validation and multi-step operation management. The BAS Server performs all data operations through its connection to the BDB Server instead of using sqlite3 directly or accessing the database file.

Tier 3: BDB Server (Bank Database Server) - operates at Tier 3 as the data tier which functions as the only system that accesses permanent storage. The server contains all necessary functions to import sqlite3 and access the database file through SQL statement execution. The system reveals its data access methods through Pyro5 RPC which keeps all database implementation specifics hidden from other system layers.

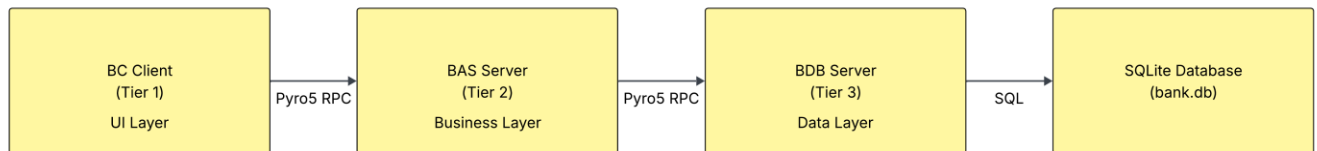


Figure 2: Three-Tier Architecture

The system design separates its components into distinct tiers which allows developers to scale and test and modify each section independently. The database schema can be modified through changes to method signatures which do not impact the BAS Server's business operations. The system allows users to switch between web-based GUI and command-line interface (CLI) without requiring any modifications to the BAS or BDB Servers. The system follows a fundamental design rule of distributed systems through its modular structure.

4.2 Component Communication Patterns

All inter-component communication uses synchronous RPC via Pyro5:

BC Client ↔ BAS Server Communication:

- Protocol: Pyro5 RPC over TCP/IP
- Pattern: Synchronous request-response
- Marshalling: Pyro5 automatic serialization of Python objects
- Connection: Client creates Pyro5 proxy to BAS Server via name server lookup
- The system includes five methods which perform the following functions: login(), get_balance(), transfer(), get_transfer_status(), request_csv_export().

BAS Server ↔ BDB Server Communication:

- Protocol: Pyro5 RPC over TCP/IP
- Pattern: Synchronous request-response
- Connection: BAS Server creates Pyro5 proxy to BDB Server via name server lookup

The system uses synchronous communication to achieve immediate operation confirmation which meets the banking industry's need for instant confirmation. During a transfer request, the client blocks synchronously while the BAS Server authenticates the request and coordinates with the BDB Server to execute the transfer atomically before returning a success or failure response. The BDB Server operates as a direct system dependency which causes all data-related operations to stop when it becomes unavailable although the BC Client and BAS Server processes continue to run. But it enables essential benefits for user authentication and error detection and transaction rule management.

4.3 Authentication and Session Management

The authentication workflow demonstrates different authorization levels which operate through its multi-level system structure.

The authentication workflow requires the BC Client to transmit credentials to BAS Server which performs authentication by checking these credentials against BDB Server user database information. The BAS Server system produces a UUID4 session token which it stores in memory before sending it to the client for future request authentication.

The BAS Server maintains session tokens in its memory system instead of saving them to the database. The design selection chooses basic design elements because they suit an educational environment. Production systems implement distributed session stores which use Redis as an example to support multiple application servers and preserve session data when the system restarts (Kleppmann, 2017).

4.4 Transfer Processing Workflow

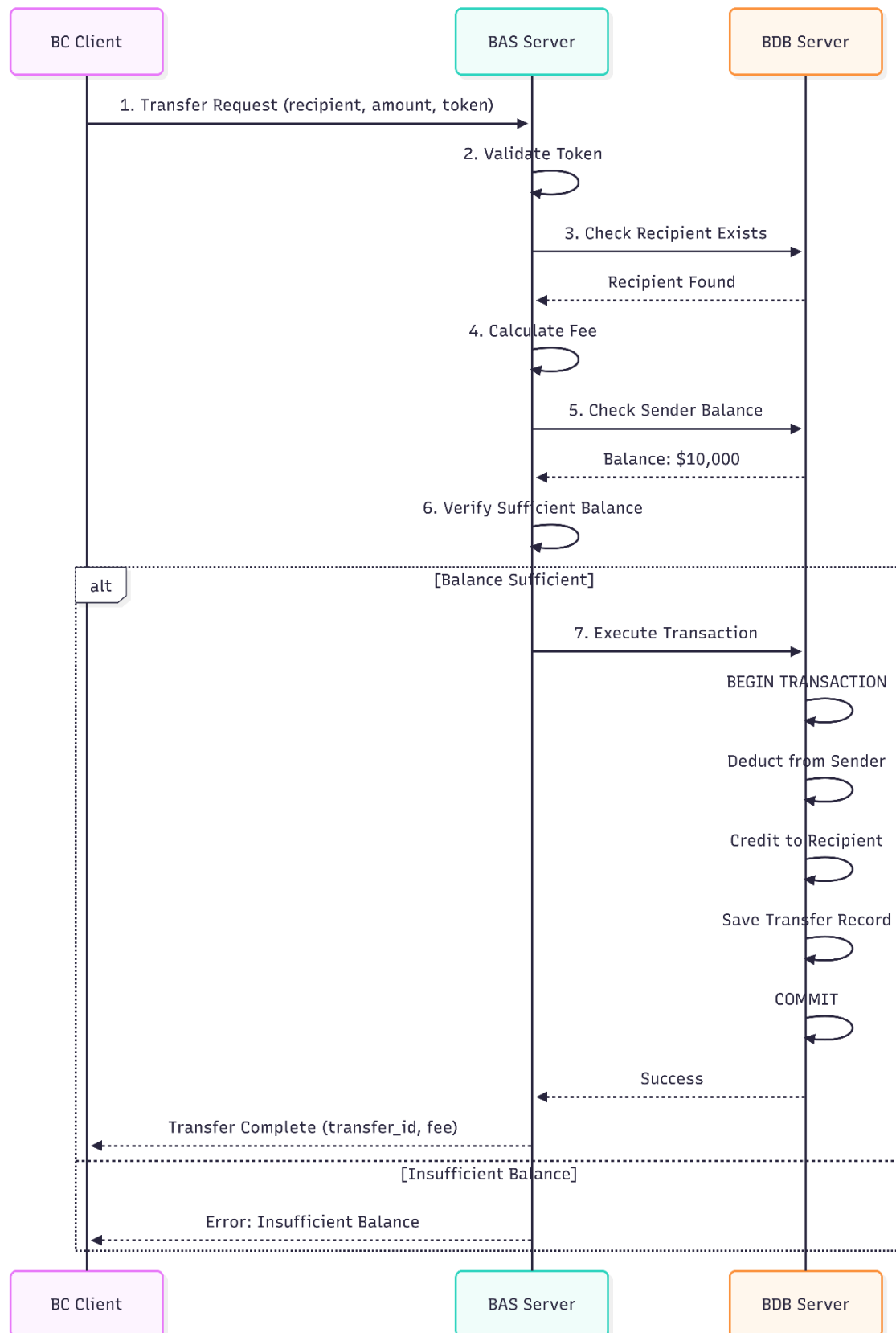


Figure 3: Transfer Processing Workflow

4.5 Database Schema and Persistence

The BDB Server manages three tables in the SQLite database:

users table:

```
user_id TEXT PRIMARY KEY (e.g., "user_001")
username TEXT UNIQUE NOT NULL
password TEXT NOT NULL (plain text for academic
purposes)
```

accounts table:

```
account_id TEXT PRIMARY KEY (e.g., "acc_001")
user_id TEXT NOT NULL (foreign key to users)
balance REAL NOT NULL (stored as floating point,
rounded to 2 decimals)
```

transfers table:

```
transfer_id TEXT PRIMARY KEY (UUID4 format)
sender_id TEXT NOT NULL (foreign key to users)
recipient_id TEXT NOT NULL (foreign key to users)
amount REAL NOT NULL
fee REAL NOT NULL
status TEXT NOT NULL (currently always
"COMPLETED")
message TEXT (optional transfer message)
timestamp TEXT NOT NULL (ISO 8601 format)
```

4.6 Consistency and Transaction Management

SQLite maintains strong consistency through its database system which supports ACID transactions. The `execute_transfer()` method in BDB Server performs all database modifications through an explicit transaction.

```
BEGIN TRANSACTION
  -- Read current balances
  -- Validate sufficient funds (defense in
depth)
  -- Update sender balance (deduct amount +
fee)
  -- Update recipient balance (add amount)
  -- Insert transfer record
COMMIT TRANSACTION
```

The system will stop its operation when any step fails because of database constraint violations or when the account balance becomes insufficient. The system will automatically perform a **ROLLBACK** to maintain atomicity. The default isolation level of SQLite which is **SERIALIZABLE** prevents multiple transactions from seeing any intermediate stages of database changes. The system uses row-level locking during SELECT operations to stop race conditions which would happen when two transfers try to access the same initial balance value before any update process.

This approach is a tradeoff between availability and consistency. If the BDB Server is no longer available, then no transfers can be processed. In the framework of CAP theorem (Brewer, 2012), such a system focuses on Consistency and Partition tolerance and compromises Availability, which is suitable for financial systems where erroneous balances are unacceptable.

4.7 Duplicate Request Handling

The system will attempt to transfer data again when network timeouts occur. The system would perform duplicate transfers when users performed their actions because it lacked duplicate detection functionality. The system contains basic functions which stop duplicate entries from being entered into the system.

- BAS Server generates `transfer_id` before processing
- BAS Server queries BDB Server: does this `transfer_id` exist in transfers table?
- If yes, reject as duplicate (prevents double-processing)
- If no, proceed with transfer execution
- The system needs to include the `transfer_id` field because it performs database insertion of transfer records.

The method operates with a basic design which differs from complex idempotency systems that maintain idempotency keys outside of business data storage (Kleppmann, 2017). The system demonstrates academic concepts effectively through its design which prevents excessive complexity. The production systems would implement client-supplied idempotency keys to perform retry operations following server restarts.

5. Implementation Procedure

5.1 Phase 1: Two-Tier Development

The proposed implementation approach resulted in Phase 1 establishing basic system functionality through a two-tier system which integrated BC Client with BAS Server and stored data in memory. The team checked business rules and communication methods at this point before they added database intricacies to the system.

Phase 1 Scope:

- BC Client with text-based menu interface
- BAS Server with in-memory Python dictionaries for data storage
- Pyro5 RPC communication between client and server
- The system contains all necessary business operations which include user verification and payment amount determination and transaction execution.
- All functional requirements except persistence.

Phase 1 Benefits:

- Rapid development without database schema complexity
- The system allows for basic testing because its memory system maintains a predictable state which remains constant.
- The system requires RPC communication functionality while maintaining proper operation of business logic operations.
- Easy state reset between test runs

The in-memory implementation used Python dictionaries:

- *self.users: Dict[str, str]* for username → password mappings
- *self.accounts: Dict[str, float]* for user_id → balance mappings
- *self.transfers: List[Dict]* for transfer records
- *self.sessions: Dict[str, str]* for token → user_id mappings

The business logic and fee calculations and RPC communication in Phase 1 operated as their designers had intended. The project followed an iterative approach to identify and resolve logical errors before it implemented database transaction complexity in the system.

5.2 Phase 2: Three-Tier Extension with Database

The BDB Server operated as a standalone database server during Phase 2 because it moved from using memory-based storage to SQLite database management system. The system required exact programming changes to maintain its separate tiers which followed their defined principles.

1. Created BDB Server (bdb_server.py):

- ONLY file that imports sqlite3
- Manages database connection and cursor operations
- Exposes data access methods via Pyro5 RPC
- Implements transaction management for transfers
- Provides CSV export functionality

2. Refactored BAS Server:

- REMOVED all in-memory data dictionaries (users, accounts, transfers)
- KEPT in-memory sessions dictionary (session state, not business data)
- ADDED Pyro5 connection to BDB Server
- MODIFIED all data access to call BDB Server methods instead of local dictionaries
- RETAINED all business logic (fee calculation, validation)

3. BC Client (Unchanged):

- No modifications required (benefit of tier separation)
- Still connects to BAS Server with identical method signatures
- Demonstrates architectural benefit: database tier added transparently to clients

Connection Management Strategy:

The BAS Server creates a Pyro5 proxy to the BDB Server during initialization. The Pyro5 threading system has a hidden requirement which states that Pyro5 proxies need to exist and function under the same thread. The solution creates fresh proxies per request thread using a helper method `_get_bdb_server()` that returns a new proxy instance. The system protects against "calling thread is not the owner" errors which happen when different threads execute their operations at the same time.

5.3 Development Challenges and Solutions

Challenge 1: Pyro5 Threading Errors

Problem: occurred when Pyro5 proxy objects were used between different threads because it produced "calling thread is not the owner" exceptions.

Solution: The system needs to create a proxy for every request through the stored URI instead of keeping a proxy object in storage.

Challenge 2: SQLite Connection Management

Problem: SQLite connections cannot be shared across threads

Solution: Create new connection for each database operation, close in finally block

Challenge 3: Tier Separation Enforcement

Problem: The BAS Server prevents developers from obtaining database authorization which they need to perform their refactoring work.

Solution: Code review to verify NO sqlite3 import in BAS Server, all SQL in BDB Server only

Challenge 4: Transaction Atomicity

Problem: The multi-step transfer process needed all its steps to run at once because it did not have any feature which would let the process stop.

Solution: Explicit BEGIN TRANSACTION, perform all operations, COMMIT or ROLLBACK in exception handler

6. User Manual

6.1 System Requirements

Software Requirements:

- Python 3.8 or higher
- Pyro5 library (install via `pip install Pyro5`)
- The system requires Windows or Linux or macOS operating system which supports Python execution.

Hardware Requirements:

- Minimal: Any modern computer capable of running Python
- Storage: <100 MB for system and database files
- Network: Localhost loopback interface (no external network required)

6.2 Installation and Setup

Step 01: Install Python Dependencies

Open a terminal or command prompt —→ `pip install Pyro5`

Verify installation —→ `python -c "import Pyro5; print('Pyro5 installed successfully')"`

Step 2: Extract Project Files

Correct project Folder structure:

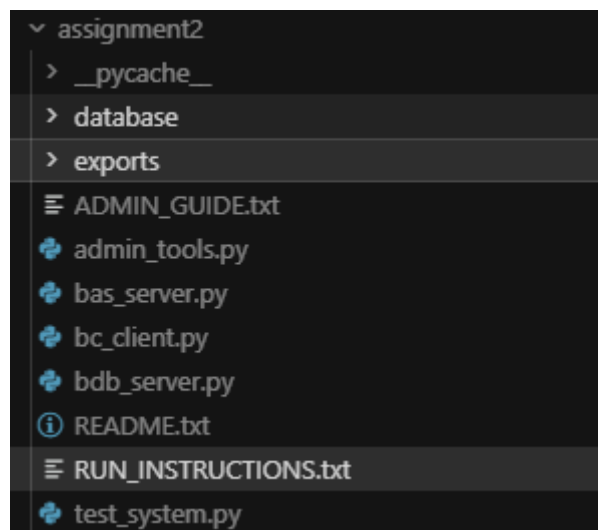


Figure 4: Project Folder Structure

The main files are:

- **bc_client.py**
- **bas_server.py**
- **bdb_server.py**
- /database/ folder (created automatically)
- /exports/ folder (created automatically)

6.3 Starting the System

To run the system, it requires Four terminal windows opened.

Terminal 1 – Run Pyro5 Name Server: **python -m Pyro5.nameserver**

Expected Output:

```
PS D:\ACBT\Year_03\Distributed Systems\Assignment 02\Banking System\assignment2> python -m Pyro5.nameserver
Not starting broadcast server for IPv6.
NS running on localhost:9090 (::1)
URI = PYRO:Pyro.NameServer@localhost:9090
```

Figure 5: Terminal 1

Terminal 2 - Start BDB Server - Navigate to project directory, then: **python bdb_server.py**

Expected Output:

```
PS D:\ACBT\Year_03\Distributed Systems\Assignment 02\Banking System\assignment2> python bdb_server.py
2026-01-18 19:17:52,935 - __main__ - INFO - Database tables created/verified
2026-01-18 19:17:52,935 - __main__ - INFO - Database already contains 2 user(s)
2026-01-18 19:17:52,935 - __main__ - INFO - Banking Database Server initialized
2026-01-18 19:17:57,458 - __main__ - INFO - Banking Database Server started and registered as 'banking.database'
2026-01-18 19:17:57,458 - __main__ - INFO - Server URI: PYRO:BankingDatabaseServer@localhost:60529
2026-01-18 19:17:57,458 - __main__ - INFO - Database location: D:\ACBT\Year_03\Distributed Systems\Assignment 02\
Banking System\assignment2\database\bank.db
2026-01-18 19:17:57,459 - __main__ - INFO - Server is ready to accept connections. Press Ctrl+C to stop.
2026-01-18 19:17:57,459 - __main__ - INFO - Note: Automatic CSV export will be attempted on shutdown (best effort
).
```

Figure 6: Terminal 2

Terminal 3 - Start BAS Server in project directory: **python bas_server.py**

Expected Output:

```
PS D:\ACBT\Year_03\Distributed Systems\Assignment 02\Banking System\assignment2> python bas_server.py
2026-01-18 19:19:47,321 - __main__ - INFO - Connected to Banking Database Server (Tier 3)
2026-01-18 19:19:47,322 - __main__ - INFO - Banking Application Server initialized
2026-01-18 19:19:51,846 - __main__ - INFO - Banking Application Server started and registered as 'banking.applica
tion'
2026-01-18 19:19:51,847 - __main__ - INFO - Server URI: PYRO:BankingApplicationServer@localhost:60547
2026-01-18 19:19:51,848 - __main__ - INFO - Server is ready to accept connections. Press Ctrl+C to stop.
```

Figure 7: Terminal 3

Terminal 4 – Start BC Client in project directory: `python bc_client.py`

Expected Output:

```
PS D:\ACBT\Year_03\Distributed Systems\Assignment 02\Banking System\assignment2> python bc_client.py - interface

=====
BANKING CLIENT - Phase 2 Final Submission
Three-Tier Architecture
=====
2026-01-19 14:30:38,481 - __main__ - INFO - Connected to Banking Application Server (Tier 2)

=====
BANKING SYSTEM - MAIN MENU
=====

1. Login
2. Exit
=====

Enter your choice: █
```

Figure 8: Terminal 4

6.4 Using the Banking System

Login process:

- In the main menu choose option 1 to login.
- Enter default user credentials:
 - user 1 : Username = `alice` | Password: `pass123`
 - user 2 : Username = `bob` | password `pass456`

```
=====
BANKING SYSTEM - MAIN MENU
=====

1. Login
2. Exit
=====

Enter your choice: 1

--- LOGIN ---
Username: alice
Password: pass123

✓ Login successful
Session token: 5631a74a...

=====
BANKING SYSTEM - MAIN MENU
=====

Logged in as: alice

1. View Balance
2. Transfer Money
3. Check Transfer Status
4. Export Database to CSV
5. Logout
6. Exit
=====

Enter your choice: █
```

Figure 9: Successful login screen showing session token

Checking Balance:

- Choose option "1" to check the current balance

```
=====
BANKING SYSTEM - MAIN MENU
=====
Logged in as: alice

1. View Balance
2. Transfer Money
3. Check Transfer Status
4. Export Database to CSV
5. Logout
6. Exit
=====

Enter your choice: 1

✓ Current balance: $9984.00
Your current balance: $9984.00
```

Figure 10: View Balance

Transferring Money:

- Choose option "2" to Transfer Money
- Enter Recipient username
- Enter amount
- Enter optional message
- System will display the transaction confirmation and the transaction ID with fee details

```
=====
BANKING SYSTEM - MAIN MENU
=====
Logged in as: alice

1. View Balance
2. Transfer Money
3. Check Transfer Status
4. Export Database to CSV
5. Logout
6. Exit
=====

Enter your choice: 2

--- TRANSFER MONEY ---
Recipient username: bob
Amount ($): 3000
Transfer message (optional): Transfer money

✓ Transfer completed successfully. Fee: $7.50
Transfer ID: 791357f0-d77d-4dcf-8659-8160316f38fa
Amount transferred: $3000.00
Fee charged: $7.50
Total deducted: $3007.50
```

Figure 11: Successful transfer showing amount, fee, and total deducted

Checking Transfer Status:

- Choose option “3” to check transfer status
- Enter transfer ID
- System displays transfer details

```
=====
BANKING SYSTEM - MAIN MENU
=====
Logged in as: alice

1. View Balance
2. Transfer Money
3. Check Transfer Status
4. Export Database to CSV
5. Logout
6. Exit
=====

Enter your choice: 3

--- CHECK TRANSFER STATUS ---
Transfer ID: 791357f0-d77d-4dcf-8659-8160316f38fa

✓ Transfer status: COMPLETED

Transfer Details:
Transfer ID: 791357f0-d77d-4dcf-8659-8160316f38fa
Status: COMPLETED
Amount: $3000.00
Fee: $7.50
Sender ID: user_001
Recipient ID: user_002
Message: Transfer money
Timestamp: 2026-01-18T19:42:11.071499
```

*Figure 12: Checking transfer status***Exporting Database to CSV:**

- Choose option “4” to Export database to CSV file
- Type “yes”
- System exports users.csv, accounts.csv, transfers.csv to /exports folder

```
=====
BANKING SYSTEM - MAIN MENU
=====
Logged in as: alice

1. View Balance
2. Transfer Money
3. Check Transfer Status
4. Export Database to CSV
5. Logout
6. Exit
=====

Enter your choice: 4

--- EXPORT DATABASE TO CSV ---
Export all database tables to CSV? (yes/no): yes

✓ Export successful. Files saved to D:\ACBT\Year_03\Distributed Systems\Assignment 02\Banking System\assignment2\
exports: users.csv (2 rows), accounts.csv (2 rows), transfers.csv (6 rows)
Files exported to the /exports directory:
- users.csv
- accounts.csv
- transfers.csv
```

Figure 13: CSV export confirmation message

Logging Out:

- Choose option “5” to logout
- The Session token deleted, returns to main menu

7. Test Cases and Validation**Test Case 1: Authentication - Valid Credentials**

- Input : Username: alice, Password: pass123
- Expected Output: Login success message, session token returned. Verification passed

Test Case 2: Authentication - Invalid Credentials

- Input : Username: alice, Password: pass123
- Expected Output: Login failed message "Invalid username or password", no token. Verification failed

```
=====
BANKING SYSTEM - MAIN MENU
=====

1. Login
2. Exit
=====

Enter your choice: 1

--- LOGIN ---
Username: alice
Password: 123456789

X Invalid username or password
```

Figure 14: Error message for invalid credentials

Test Case 3: Balance Query

- Input : Select "View Balance" menu option
- Expected Output: Display balance: (e.g., \$5000)

Test Case 4: Transfer - Free Tier

Preconditions: User A balance : \$5,000, User B balance : \$2,000

- Input: User A Transfers \$1500
- Expected Output:
 - Fee: \$0
 - User A balance : \$3,500
 - User B balance : \$3,500

Test Case 5 : Transfer Entry Tier

- Input: recipient name, amount (\$6000)
- Expected Output: Transferred money to “alice” with \$15.00 fee which is 0.25%

```

--- TRANSFER MONEY ---
Recipient username: alice
Amount ($): 6000
Transfer message (optional):

✓ Transfer completed successfully. Fee: $15.00
Transfer ID: d770d45b-13b6-4b2d-8eb3-dfb2542e1a3a
Amount transferred: $6000.00
Fee charged: $15.00
Total deducted: $6015.00

```

Test Case 6 : Transfer Top Tier

Preconditions: User E balance : \$200,000, User F balance : \$50,000

- Input: User E Transfers \$150,000
- Expected Output:
 - Fee: \$100
 - User E balance : \$49,900
 - User F balance : \$ \$200,000

Test Case 7 : Insufficient Balance

Preconditions: User balance : \$4,000

- Input: User G Transfers \$5,000
- Expected Output:
 - Fee: 0
 - User G balance : \$4,000 (unchanged)

```

BANKING SYSTEM - MAIN MENU
=====
Logged in as: bob

1. View Balance
2. Transfer Money
3. Check Transfer Status
4. Export Database to CSV
5. Logout
6. Exit
=====

Enter your choice: 2

--- TRANSFER MONEY ---
Recipient username: alice
Amount ($): 5000
Transfer message (optional):

X Insufficient balance. Required: $5012.50 (amount: $5000.00 + fee: $12.50), Available: $1986.00
Fee that would be charged: $12.50

```

Test Case 8 : Invalid Recipient when transferring

- Input: Recipient: Charlie, Amount
- Expected Output: Transfer rejected with "Recipient 'charlie' not found" error, No money deducted from sender.

```
=====
BANKING SYSTEM - MAIN MENU
=====
Logged in as: bob

1. View Balance
2. Transfer Money
3. Check Transfer Status
4. Export Database to CSV
5. Logout
6. Exit
=====

Enter your choice: 2

--- TRANSFER MONEY ---
Recipient username: Charlie
Amount ($): 500
Transfer message (optional):

X Recipient 'Charlie' not found
```

Figure 15: Invalid recipient

Test Case 9 : Transfer to Self

- Preconditions: user logged into bob's account
- Input: enter recipient as 'bob'
- Expected output: error message "Cannot transfer money to yourself"

```
=====
BANKING SYSTEM - MAIN MENU
=====
Logged in as: bob

1. View Balance
2. Transfer Money
3. Check Transfer Status
4. Export Database to CSV
5. Logout
6. Exit
=====

Enter your choice: 2

--- TRANSFER MONEY ---
Recipient username: bob
Amount ($): 500
Transfer message (optional):

X Cannot transfer money to yourself
```

8. Discussion and Critical Analysis

8.1 Design Decisions and Justifications

The system depends on synchronous RPC communication which enables immediate transaction verification because banking operations need users to get fast success or failure responses (e.g. when funds are insufficient). The system fault tolerance methods Nygard (2018) presented for asynchronous patterns introduce time delays which oppose the requirement for immediate system verification.

Pyro5 was selected over gRPC for its Pythonic API and minimal setup. The method invocation of Pyro5 operates without need for protocol-buffer definitions or code generation which makes it suitable for academic environments because it enables students to study RPC fundamentals without dealing with performance optimization challenges.

The system prevents duplicate transfers through a transfer_id verification process which occurs before any processing takes place. The system demonstrates idempotency through its lightweight design which teaches students about this concept while avoiding production-level framework complexity according to Kleppmann (2017).

8.2 Trade-offs and Limitations

The system prioritizes consistency over availability (CP in CAP theorem), accepting that database unavailability prevents all transfers (Brewer, 2012). The method serves financial systems because any wrong entry in the balance would result in unacceptable errors.

The system faces **three** main restrictions which include:

- (1) The system lacks a stress testing mechanism to handle multiple transactions at once
- (2) The system loses all session data from memory when it restarts
- (3) The system only accepts plain text passwords for educational use.

8.3 Lessons Learned

The Phase 1→Phase 2 iterative method succeeded because it enabled business logic testing before developers started working on complex database functionality. The Pyro5 threading constraint revealed that remote proxies differ from local objects, requiring fresh proxy creation per request (Waldo et al.,1997). The testing process revealed that error messages should display both exact dollar amounts and complete system failure cause information.

9. Conclusion

The system achieved its goals by creating a three-tier distributed banking system which performed remote procedure calls and demonstrated tier separation and transaction management and consistency guarantee functionality. The system functioned as designed because it allowed users to verify their identity and performed account balance checks and processed payments with integrated fee management and transfer status monitoring while keeping all system elements completely independent.

The development process of Phase 1 to Phase 2 used an iterative approach to test business rules before adding database complexity to the system. The system achieves technical success through its implementation of synchronous RPC which delivers instant confirmation and its six-tier progressive fee system which ranges from 0% to 0.05% and its ability to handle duplicate requests and its detailed audit system which uses structured logging and CSV export for data retrieval.

The testing process verified all requirements through manual human assessment and automated system verification which proved the correct operation of authentication sequences and fee calculation and error handling and data storage precision. Financial operations need absolute accuracy in their processes so the system gives priority to maintaining consistency instead of availability.

The academic version of the architecture presents distributed systems principles through its simplified design. The system maintains its ability to update its database structure and add new user interfaces and perform background payment processing through its current business operations. The project enables me to gain practical knowledge about distributed system communication behavior and distributed system consistency needs and separate system architecture design.

10. References

- Birrell, A. D., & Nelson, B. J. (1984). Implementing remote procedure calls. *ACM Transactions on Computer Systems*, 2(1), 39–59. <https://doi.org/10.1145/2080.357392>
- Brewer, E. A. (2012). CAP twelve years later: How the “rules” have changed. *Computer*, 45(2), 23–29. <https://doi.org/10.1109/MC.2012.37>
- Google. (n.d.). *gRPC: A high-performance, open source universal RPC framework*. Retrieved April 12, 2024, from <https://grpc.io/>
- Kleppmann, M. (2017). *Designing data-intensive applications: The big ideas behind reliable, scalable, and maintainable systems*. O'Reilly Media.
- Nygard, M. T. (2018). *Release it! Design and deploy production-ready software* (2nd ed.). Pragmatic Bookshelf.
- Pyro5. (n.d.). *Pyro5: Python remote objects*. Retrieved April 12, 2024, from <https://pyro5.readthedocs.io/>
- Richardson, C. (2018). *Microservices patterns: With examples in Java*. Manning Publications.
- SQLite Consortium. (n.d.). *SQLite documentation*. Retrieved April 12, 2024, from <https://www.sqlite.org/docs.html>
- Tanenbaum, A. S., & Van Steen, M. (2017). *Distributed systems: Principles and paradigms* (3rd ed.). Pearson Education.
- Waldo, J., Wyant, G., Wollrath, A., & Kendall, S. (1997). A note on distributed computing. In J. Vitek & C. Tschudin (Eds.), *Mobile object systems: Towards the programmable Internet* (pp. 49–64). Springer. <https://doi.org/10.1007/BFb0054206>

AI Tool Declaration: This project was written with the help of Claude (Anthropic) on January 19, 2026, to receive guidance with research, assistance in structuring and verification of the reference formatting